

GRADUAÇÃO

 FIAP

MOBILE APPLICATION DEVELOPMENT

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS



AULA 09

Salvando dados em um BaaS

Criação de documentos com Cloud Firestore



“Cada acesso a cada objeto precisa ser verificado quanto à autoridade.”
— *Jerome H. Saltzer e Michael D. Schroeder, tradução livre*



JÁ SABEMOS SALVAR DADOS REMOTAMENTE

```
ação do usuário
  ↓
useMutation
  ↓
mutationFn
  ↓
POST /posts
  ↓
API
```

AGORA O MECANISMO DE ACESSO AO BACKEND SERÁ OUTRO.

O QUE MUDA SEM UMA API CONSTRUÍDA PELA EQUIPE?

O aplicativo continuará executando uma escrita remota, mas passará a usar diretamente um serviço de BaaS.

OBJETIVOS

Ao final da aula, você será capaz de:

- comparar a escrita por API própria com a escrita em um BaaS;
- explicar o contrato de `addDoc` ;
- distinguir ID automático, `serverTimestamp` e `DocumentReference` ;
- explicar como Security Rules autorizam uma criação;
- integrar a escrita do Firestore com uma mutation já conhecida;
- justificar a invalidação das consultas afetadas.

API PRÓPRIA E BACKEND AS A SERVICE

API PRÓPRIA

```
aplicativo
  ↓ POST /posts
nossa API
  ↓ autorização
banco de dados
```

CLOUD FIRESTORE

```
aplicativo
  ↓ chamada ao SDK
Cloud Firestore
  ↓ Security Rules
documentos
```

A RESPONSABILIDADE MUDOU DE LUGAR

Com API própria	Com Cloud Firestore
A equipe implementa o endpoint	O SDK acessa um serviço existente
A API recebe a solicitação	O Firestore recebe a solicitação
O servidor valida a autorização	Security Rules validam a autorização
O servidor persiste no banco	O Firestore persiste o documento

DESTINO, CONTRATO, IDENTIDADE E FALHAS CONTINUAM FAZENDO PARTE DA ESCRITA.

SDK E SERVIÇO REMOTO

O pacote `firebase/firestore` executa dentro da aplicação.

```
aplicativo
└─ SDK
    ↓ rede
    Cloud Firestore
    ↓
    documentos
```

O SDK permite que a aplicação:

- crie referências para locais do banco;
- descreva os dados de uma escrita;
- envie a operação ao serviço remoto;
- acompanhe o resultado por uma Promise;
- utilize o estado local mantido pelo cliente Firestore.

O SDK DESCREVE E TRANSPORTA A OPERAÇÃO. O SERVIÇO REMOTO DECIDE E PERSISTE.

COMO INDICAMOS ONDE O DOCUMENTO SERÁ CRIADO?

Uma escrita começa pela localização que receberá o novo documento.

REFERÊNCIA DA COLEÇÃO

```
import { collection } from "firebase/firestore";  
  
const postsCollection = collection(db, "posts");
```

`postsCollection` representa o caminho:

`/posts`

Nenhuma operação remota foi executada.

Elemento	Responsabilidade
<code>collection(db, "posts")</code>	Representar o destino
<code>addDoc(...)</code>	Solicitar a criação de um documento
<code>DocumentReference</code>	Representar o documento criado

REFERENCIAR UM LOCAL NÃO CRIA DADOS NESSE LOCAL.

O CONTRATO DE ADDDOC

```
addDoc(referenciaDaColecao, dadosDoDocumento)
```

Resultado assíncrono:

```
Promise<DocumentReference>
```

Parte	Pergunta respondida
CollectionReference	Em qual coleção criar?
data	Quais campos armazenar?
Promise	Quando a operação terminará?
DocumentReference	Onde ficou o documento criado?

CRIANDO UM DOCUMENTO

```
import { addDoc } from "firebase/firestore";

const documentReference = await addDoc(postsCollection, {
  content: "Estudando bancos orientados a documentos",
  authorId: user.uid,
});
```

postsCollection → onde criar
objeto → o que armazenar

Cada execução de `addDoc` solicita a criação de um novo documento.

IDENTIDADE E CAMPOS

Depois da criação, o documento possui duas partes:

```
caminho
  posts/7qA2fLm9

campos
  content: "Estudando bancos orientados a documentos"
  authorId: "user-42"
```

O IDENTIFICADOR PERTENCE AO CAMINHO DO DOCUMENTO.

`addDoc` atribui esse identificador automaticamente:

```
posts
├── 7qA2fLm9
├── bK8pQ3xN
└── mR5tV9zC
```

A aplicação não precisa escolher um ID para cada criação.

O ID NÃO FAZ PARTE DOS CAMPOS

```
documentReference.id;
```

```
document.data();
```

Origem	Conteúdo
Referência ou snapshot	Identificador
<code>document.data()</code>	Campos armazenados

O Firestore não duplica automaticamente o ID dentro dos dados.

ID AUTOMÁTICO NÃO REPRESENTA ORDEM

O ID distingue documentos, mas não informa qual deles foi criado primeiro.

O modelo precisa de um campo próprio para ordenar pelo momento da criação:

```
posts/7qA2fLm9
├── content
├── authorId
└── createdAt
```

```
orderBy("createdAt", "desc");
```

IDENTIDADE E ORDEM RESOLVEM PROBLEMAS DIFERENTES.

O RELÓGIO DO DISPOSITIVO

Uma primeira tentativa poderia usar:

```
createdAt: new Date()
```

Esse valor depende do dispositivo:

- o relógio pode estar incorreto;
- o usuário pode alterar o horário;
- dispositivos podem discordar entre si.

Ô cliente não deve definir sozinho o horário usado como referência comum.

SERVERTIMESTAMP

```
import { serverTimestamp } from "firebase/firestore";

await addDoc(postsCollection, {
  content: "Estudando bancos orientados a documentos",
  authorId: user.uid,
  createdAt: serverTimestamp(),
});
```

`serverTimestamp()` solicita que o serviço determine o horário da escrita.

UMA INSTRUÇÃO PARA O SERVIDOR

`serverTimestamp()` devolve um `FieldValue` especial.

```
cliente envia
  createdAt: serverTimestamp()
```

```
servidor resolve
  createdAt: Timestamp
```

O VALOR ENVIADO REPRESENTA UMA TRANSFORMAÇÃO, NÃO O HORÁRIO FINAL.

Momento	Valor
Objeto de escrita	<code>FieldValue</code> de <code>serverTimestamp</code>
Documento persistido	<code>Timestamp</code>
Documento lido	<code>Timestamp</code>

ESCRITA COMPLETA

```
import {
  addDoc,
  collection,
  serverTimestamp,
} from "firebase/firestore";

const postsCollection = collection(db, "posts");

const documentReference = await addDoc(postsCollection, {
  content: "Estudando bancos orientados a documentos",
  authorId: user.uid,
  createdAt: serverTimestamp(),
});
```

O QUE RECEBEMOS DEPOIS DA CRIAÇÃO?

A escrita produz uma referência para o novo documento.

DOCUMENTREFERENCE

```
const documentReference = await addDoc(  
  postsCollection,  
  data,  
);
```

A Promise resolve com uma `DocumentReference`.

```
documentReference.id;  
// "7qA2fLm9"  
  
documentReference.path;  
// "posts/7qA2fLm9"
```

A referência identifica precisamente a localização criada.

A REFERÊNCIA NÃO É UM SNAPSHOT

A `DocumentReference` não contém automaticamente:

- uma cópia completa dos campos;
- o valor final de `createdAt`;
- um `DocumentSnapshot`;
- a coleção atualizada;
- o cache atualizado do TanStack Query.

REFERÊNCIA APONTA PARA O DADO. SNAPSHOT CONTÉM O ESTADO LIDO.

CONFIRMAÇÃO REMOTA

A Promise de `addDoc`:

- resolve quando o backend confirma a criação;
- rejeita quando o backend informa uma falha;
- pode permanecer pendente enquanto o backend não puder ser alcançado.

```
pending
├─ resolve → documento confirmado
└─ reject  → criação recusada
```

No `useMutation`, `isPending` acompanha essa Promise.

COMPENSAÇÃO DE LATÊNCIA

O SDK pode incorporar a escrita ao estado local antes da confirmação remota.

aplicação local da escrita
≠
confirmação pelo backend

Se o cliente estiver sem conexão, o estado local pode refletir a criação enquanto a Promise continua pendente.

QUEM AUTORIZA UMA ESCRITA ENVIADA DIRETAMENTE PELO APLICATIVO?

O CLIENTE NÃO É UMA FRONTEIRA CONFIÁVEL

O código da aplicação está sob controle do dispositivo.

Um cliente modificado pode:

- ignorar a validação do formulário;
- alterar os campos enviados;
- substituir um identificador de usuário;
- chamar o SDK fora do fluxo previsto pela interface.

O `firebaseConfig` identifica o projeto, mas não autoriza operações.

A AUTORIZAÇÃO PRECISA ACONTECER NO SERVIÇO REMOTO POR MEIO DAS SECURITY RULES.

CAMINHO DE UMA SOLICITAÇÃO

```
addDoc
  ↓
solicitação autenticada
  ↓
Security Rules
  └─ permitida → documento criado
  └─ negada    → erro devolvido
```

As regras são avaliadas pelo serviço antes da persistência.

OPERAÇÃO CREATE

As regras distinguem operações diferentes:

```
get
list
create
update
delete
```

Para autorizar apenas novos documentos:

```
allow create: if request.auth != null;
```

`allow write` também incluiria atualização e exclusão.

Quando Firebase Authentication identifica a pessoa, `request.auth` contém os dados dessa identidade.

```
request.auth.uid
```

O UID pode participar da decisão de autorização.

ESTADO PROPOSTO

```
request.resource.data
```

Durante uma criação, esse objeto representa os campos que o cliente está tentando persistir.

```
request.resource.data
├── content
├── authorId
└── createdAt
```

Os dados ainda precisam ser avaliados pelas regras.

CAMPOS E TIPOS ESPERADOS

```
request.resource.data.keys().hasAll([  
  "content", "authorId", "createdAt"  
])
```

`hasAll` exige os campos obrigatórios.

```
request.resource.data.keys().hasOnly([  
  "content", "authorId", "createdAt"  
])
```

`hasOnly` rejeita campos não previstos.

```
request.resource.data.content is string
```

```
request.resource.data.content.size() > 0
```

```
request.resource.data.content.size() <= 280
```

As regras também verificam tipos e valores.

AUTOR E HORÁRIO

IDENTIDADE

```
request.resource.data.authorId  
== request.auth.uid
```

O autor precisa ser a pessoa autenticada.

TEMPO

```
request.resource.data.createdAt  
== request.time
```

O horário precisa ter sido produzido pelo servidor.

CONTRATO DOS CAMPOS

```
function hasExpectedFields() {  
  return request.resource.data.keys().hasAll([  
    "content", "authorId", "createdAt"  
  ])  
  && request.resource.data.keys().hasOnly([  
    "content", "authorId", "createdAt"  
  ]);  
}  
  
function hasValidContent() {  
  return request.resource.data.content is string  
    && request.resource.data.content.size() > 0  
    && request.resource.data.content.size() <= 280;  
}
```

AUTORIZAÇÃO DA CRIAÇÃO

```
match /posts/{postId} {  
  allow create: if request.auth != null  
    && hasExpectedFields()  
    && hasValidContent()  
    && request.resource.data.authorId  
      == request.auth.uid  
    && request.resource.data.createdAt  
      == request.time;  
}
```

À criação somente prossegue quando todas as condições são verdadeiras.

TRÊS MECANISMOS DIFERENTES

Mecanismo	Responsabilidade
Formulário	Feedback antes do envio
TypeScript	Coerência dos tipos durante o desenvolvimento
Security Rules	Autorização e proteção do dado no serviço remoto

VALIDAÇÃO NO CLIENTE MELHORA O USO. REGRA NO BACKEND PROTEGE O DADO.

FALHA E REPETIÇÃO

Uma criação pode falhar porque:

- a pessoa não está autenticada;
- as regras recusaram os dados;
- o serviço não pode ser alcançado;
- o projeto atingiu algum limite.

Cada nova execução de `addDoc` gera outra identidade:

```
primeira chamada → posts/7qA2fLm9  
segunda chamada → posts/bK8pQ3xN
```

REPETIR UMA OPERAÇÃO INCERTA PODE PRODUZIR DOCUMENTOS DUPLICADOS.

ONDE O TANSTACK QUERY ENTRA?

O mecanismo de persistência mudou. O tipo de operação continua sendo uma mutation.

A MUTATION NÃO DEPENDE DE HTTP

Antes:

```
async function createPost(input: NewPost) {  
  return fetch(API_URL, {  
    method: "POST",  
    body: JSON.stringify(input),  
  });  
}
```

Agora:

```
async function createPost(input: NewPost) {  
  return addDoc(postsCollection, input);  
}
```

Nos dois casos, a `mutationFn` recebe variáveis e devolve uma Promise.

O TANSTACK QUERY NÃO EXIGE QUE A FUNÇÃO UTILIZE HTTP DIRETAMENTE.

SERVIÇO DE CRIAÇÃO

```
type NewPost = {  
  content: string;  
  authorId: string;  
};  
  
export async function createPost(input: NewPost) {  
  return addDoc(postsCollection, {  
    ...input,  
    createdAt: serverTimestamp(),  
  });  
}
```

O valor resolvido será a `DocumentReference`.

MUTATION DO FIRESTORE

```
const createPostMutation = useMutation({  
  mutationFn: createPost,  
  onSuccess: (documentReference) => {  
    console.log(documentReference.id);  
  },  
});
```

Quando `createPost` devolve a Promise de `addDoc`:

```
isPending  
  ↓  
backend confirma a criação  
  ↓  
onSuccess(DocumentReference)
```

EXISTEM DOIS CACHES

Cache	Conteúdo
Cliente do Firestore	Estado local usado pelo SDK
TanStack Query	Valor devolvido pela <code>queryFn</code>

cache do Firestore
≠
cache do TanStack Query

antes → Firestore: 10 | TanStack Query: 10
depois → Firestore: 11 | TanStack Query: 10

Uma escrita do SDK não reescreve automaticamente o resultado da leitura anterior.

INVALIDAÇÃO DEPOIS DA CRIAÇÃO

```
const queryClient = useQueryClient();

const createPostMutation = useMutation({
  mutationFn: createPost,
  onSuccess: async () => {
    await queryClient.invalidateQueries({
      queryKey: ["posts"],
    });
  },
});
```

A invalidação marca as leituras relacionadas como desatualizadas.

Ela não persiste o documento. Para uma query ativa, pode produzir outra execução de `getDocs`.

FLUXO COMPLETO

```
mutate(input)
  ↓
createPost(input)
  ↓
addDoc(postsCollection, data)
  ↓
Security Rules
├─ negam → Promise rejeita
├─ peritem
  ↓
documento criado
  ↓
DocumentReference
  ↓
invalidateQueries(["posts"])
  ↓
nova leitura com getDocs
```

RESPONSABILIDADES

Elemento	Responsabilidade
CollectionReference	Identificar o destino
addDoc	Solicitar uma criação com ID automático
serverTimestamp	Solicitar o horário do servidor
Security Rules	Autorizar a operação e proteger o contrato
Firestore	Persistir o documento
DocumentReference	Identificar a localização criada
useMutation	Representar o ciclo da operação
invalidateQueries	Sinalizar leituras desatualizadas

SÍNTESE

- `collection` representa o destino;
- `addDoc` cria um documento com ID automático;
- `serverTimestamp` delega o horário ao serviço;
- `DocumentReference` identifica o documento criado;
- a Promise representa a confirmação remota;
- Security Rules autorizam a criação;
- `useMutation` acompanha o ciclo da operação;
- a invalidação reconcilia as leituras afetadas.



APLICATIVO “PACEUP”

Publicando check-ins no mural com Cloud Firestore

[Acesse o Codelab](#)

O QUE VOCÊ VAI APRENDER?

- autorizar a criação com Security Rules;
- criar documentos com `addDoc` e ID automático;
- solicitar o horário com `serverTimestamp`;
- representar a escrita com `useMutation`;
- atualizar o mural depois da publicação.

REFERÊNCIAS

- [Cloud Firestore — Modelo de dados](#)
- [Cloud Firestore — Adicionar dados](#)
- [Firebase JavaScript API — Firestore](#)
- [Cloud Firestore — Primeiros passos com Security Rules](#)
- [Cloud Firestore — Condições nas regras](#)

REFERÊNCIAS

- Cloud Firestore — Controle de campos
- Firebase Rules — `request.time`
- TanStack Query — `useMutation`
- TanStack Query — Invalidação de queries
- Saltzer e Schroeder — The Protection of Information in Computer Systems

PROJETANDO APLICAÇÕES INTENSIVAS EM DADOS

MARTIN KLEPPMANN

O livro discute os fundamentos que aparecem por trás desta aula:

- modelos de dados;
- armazenamento e recuperação;
- relógios e ordenação de eventos;
- consistência em sistemas distribuídos;
- consequências de falhas parciais.

[Conheça o livro e seus materiais](#)

LICENÇA

Este material é disponibilizado sob a licença Creative Commons Atribuição 4.0 Internacional (CC BY 4.0).

  Atribuição 4.0 Internacional

Autor: Felipe Cavalcante Vilella de Oliveira, 2026